

Análise de Desempenho e Custo em Funções Serverless: Um Estudo Comparativo entre Quarkus e Go na AWS Lambda

Gabriel L. Resende

¹Instituto de Ciências Exatas e Informática
Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Belo Horizonte - MG - Brasil

`gabriel.lourenco@sga.pucminas.br`

Abstract. *Serverless computing has become a widely adopted model for cloud applications due to its infrastructure abstraction and consumption-based pricing. This work presents a comparative analysis between serverless functions developed in Go and Quarkus, executed on the AWS Lambda platform. The research follows a quantitative and experimental approach, evaluating Cold Start latency, Warm Start performance, memory consumption and cost. The experiments will be conducted in a controlled AWS environment using Terraform, GitHub Actions, K6, CloudWatch, and X-Ray. The study aims to identify efficiency and cost differences between the evaluated technologies, supporting architectural decisions for serverless applications.*

Resumo. *A computação serverless consolidou-se como um modelo amplamente utilizado em aplicações em nuvem devido à abstração da infraestrutura e ao modelo de cobrança baseado em consumo. Com isso, este trabalho apresenta uma análise comparativa entre funções serverless desenvolvidas em Go e em Quarkus, executadas na AWS Lambda. A pesquisa possui abordagem quantitativa e experimental, avaliando métricas como latência de Cold Start, desempenho em Warm Start, consumo de memória e custo. Os experimentos serão conduzidos em ambiente controlado na AWS, utilizando Terraform, GitHub Actions, K6, CloudWatch e X-Ray. Espera-se identificar diferenças de eficiência entre as tecnologias, contribuindo para decisões arquiteturais em aplicações serverless.*

Bacharelado em Engenharia de Software - PUC Minas
Trabalho de Conclusão de Curso (TCC)

Orientador de conteúdo (TCC I): Danilo Maia - `danilomaia@pucminas.br`
Orientador de conteúdo (TCC I): João Pedro Batisteli - `joabatisteli@pucminas.br`
Orientador de conteúdo (TCC I): Leonardo Vilela - `leonardocardoso@pucminas.br`
Orientador acadêmico (TCC I): Cleiton Tavares - `cleitontavares@pucminas.br`
Orientador do TCC II: (A ser definido no próximo semestre)

Belo Horizonte, 03 de Junho de 2026.

1. Introdução

A computação em nuvem revolucionou a implantação e a operação de software, destacando-se nos últimos anos a ascensão do modelo arquitetural *Serverless* e, mais

especificamente, o paradigma *Function-as-a-Service (FaaS)* [Fox et al. 2017]. Nesse modelo de abstração de infraestrutura, os desenvolvedores concentram-se exclusivamente na lógica de negócios, delegando o provisionamento, o dimensionamento e o gerenciamento de servidores para os provedores de nuvem [Hassan et al. 2021]. Plataformas como o *AWS Lambda* popularizaram esse paradigma ao oferecerem escalabilidade automática e um modelo de cobrança *pay-per-use*, onde os custos são estritamente proporcionais ao tempo de execução e à memória alocada [Eivy 2017]. Apesar das notáveis vantagens operacionais, a eficiência do consumo de recursos tornou-se um desafio crítico na engenharia de software moderna. O tempo de inicialização da *runtime* e a sobrecarga de execução ditam não apenas a latência da aplicação, mas também o seu custo financeiro, evidenciando a necessidade de uma racionalização criteriosa dos recursos na etapa de desenvolvimento e experimentação arquitetural [Wang et al. 2018].

Historicamente, linguagens compiladas nativamente, como o *Golang (Go)*, têm dominado o ecossistema *FaaS* por apresentarem tempos de inicialização rápidos e baixo consumo de memória, ideais para o ciclo de vida curto de funções *serverless*. Em contrapartida, o ecossistema *Java*, amplamente consolidado no desenvolvimento *backend* corporativo, sempre sofreu com inicializações lentas e alto consumo de memória devido à dependência da *Java Virtual Machine (JVM)*. No entanto, inovações recentes introduziram *frameworks* modernos como o *Quarkus*, que promete mitigar essas limitações por meio da compilação *Ahead-of-Time (AOT)* via *GraalVM*, gerando binários nativos otimizados. Diante dessa evolução tecnológica, surge a necessidade de investigar se essas adaptações arquiteturais tornam o *Java* competitivo frente a linguagens inerentemente mais leves na nuvem. Sendo assim, o problema central deste estudo consiste na **dificuldade de estabelecer os *trade-offs* de desempenho e custo do *Quarkus* em comparação ao *Golang* em funções *Serverless*.**

A relevância de se investigar e resolver este problema reside na necessidade de otimização de desempenho e eficiência econômica na operação de software em escala. Diante disso, em arquiteturas utilizando o modelo de *FaaS*, milissegundos adicionais de tempo de execução ou frações extras de memória configuradas, resultam em aumentos exponenciais de custos e desperdício de recursos computacionais [van Eyk et al. 2018]. Dessa forma, além do impacto financeiro, a degradação da latência oriunda de *cold starts* severos, ou seja, atrasos para iniciar e processar uma nova requisição após períodos de ociosidade, pode comprometer a experiência do usuário e inviabilizar o cumprimento de *Service Level Agreements (SLAs)* em sistemas síncronos [Männer et al. 2018]. Ao quantificar empiricamente as métricas destas duas abordagens concorrentes, este estudo contribui significativamente para a literatura de engenharia de software, fornecendo uma base de conhecimento para que arquitetos e engenheiros possam realizar decisões fundamentadas, equilibrando o reaproveitamento do conhecimento do ecossistema *Java* com os rigorosos requisitos de performance em soluções *serverless* na nuvem [Leitner et al. 2019].

Para responder à problemática levantada, o objetivo geral deste trabalho é **realizar uma análise comparativa de desempenho e custo financeiro entre funções *Serverless* desenvolvidas em *Quarkus* e em *Go*, executadas no ambiente da *AWS***. Visando o cumprimento deste propósito, foram estabelecidos os seguintes objetivos específicos: (i) avaliar o impacto do tempo de inicialização das *Lambdas* na latência de resposta; (ii) analisar o desempenho das funções em diferentes regimes de carga; (iii) comparar o impacto das

diferentes *runtimes* no custo financeiro.

Como resultados esperados da condução deste estudo, almeja-se a obtenção de uma base de métricas quantitativas que permitam avaliar o comportamento de aplicações *Quarkus* e *Go* no ecossistema *AWS Lambda* sob contextos distintos. Espera-se produzir um catálogo claro das penalidades de latência, consumo de recursos e projeção de faturamento para cada tecnologia. Com esses dados consolidados, planeja-se conceber diretrizes arquiteturais que auxiliem as equipes de desenvolvimento na escolha da melhor tecnologia em função dos requisitos não-funcionais do projeto. Além disso, todos os artefatos de código-fonte, scripts de automação e configurações de pipelines utilizados na experimentação serão tornados públicos, assegurando a transparência e a reprodutibilidade metodológica desta pesquisa.

O restante deste documento está estruturado da seguinte maneira: a Seção 2 apresenta a fundamentação teórica, esclarecendo os conceitos de *Cloud Computing*, *Serverless*, *AWS Lambda*, *Cold Start*, *Warm Start* e Compilação Nativa. Em seguida, a Seção 3 aborda uma revisão da literatura sobre os principais trabalhos correlatos. Por fim, a Seção 4 detalha a metodologia e o planejamento da pesquisa, descrevendo o ambiente computacional, as ferramentas de automação e os procedimentos de coleta de métricas.

2. Fundamentação Teórica

Esta seção apresenta os conceitos fundamentais que sustentam a análise comparativa proposta, detalhando o funcionamento da arquitetura *serverless*, as mecânicas do ciclo de vida de execução em nuvem e os diferentes paradigmas de compilação adotados pelas tecnologias avaliadas.

2.1. Computação em Nuvem e FaaS

A evolução da computação em nuvem revolucionou a implantação e a operação de software, culminando no desenvolvimento do ecossistema *serverless*, um modelo baseado no paradigma *FaaS* [Fox et al. 2017]. Neste modelo de abstração de infraestrutura, os desenvolvedores concentram-se exclusivamente na lógica de negócios, delegando o provisionamento, o dimensionamento automático e o gerenciamento de servidores para os provedores de nuvem [Hassan et al. 2021].

Plataformas consolidadas neste modelo, como o *AWS Lambda*, popularizaram esse paradigma ao oferecerem escalabilidade automática e um modelo econômico de cobrança baseado estritamente no tempo de execução e na memória alocada, configurando o padrão *pay-per-use* [Eivy 2017]. Na *AWS*, especificamente, o modelo de tarifação abandona a locação mensal de máquinas e passa a ser calculado de acordo com duas dimensões principais: o número total de requisições e o tempo de execução. Além disso, o custo dessa duração é diretamente proporcional à quantidade de memória RAM configurada para a função. Como a capacidade de processamento (CPU) e de rede escala linearmente de acordo com a memória alocada, aplicações com execuções mais rápidas e menor consumo de memória resultam em faturas expressivamente menores. Consequentemente, a eficiência do *runtime* da linguagem escolhida deixa de ser apenas uma métrica de desempenho técnico para se tornar um dos principais fatores determinante do impacto financeiro na operação do sistema.

2.2. Ciclo de Vida de Execução e Gerenciamento de Recursos

Um aspecto crítico e inerente ao ambiente de funções como serviço é o ciclo de vida da execução das funções, que impacta diretamente a latência e o custo operacional das aplicações. Este ciclo é caracterizado por dois estados operacionais distintos: *Cold Start* e *Warm Start*.

O *Cold Start* ocorre quando uma função é invocada após um período de inatividade ou para atender a uma demanda repentina de escalabilidade horizontal. Neste estado, o provedor de nuvem precisa instanciar um novo ambiente de execução do zero e inicializar o *runtime* da linguagem, o que eleva consideravelmente a latência inicial da resposta e pode comprometer o cumprimento de acordos de *SLAs* [Männer et al. 2018].

Em contrapartida, o *Warm Start* acontece quando a requisição utiliza um ambiente de execução que já foi instanciado e permanece ativo. Dessa forma, como o ambiente e o código já estão carregados na memória, o tempo de resposta é significativamente reduzido, representando o desempenho de regime estável da aplicação.

2.3. Paradigmas de Execução: Compilação Nativa e Máquinas Virtuais

O impacto do *Cold Start* está diretamente associado ao peso computacional e ao tempo necessário para carregar o ambiente e inicializar o *runtime* da linguagem escolhida. Linguagens compiladas nativamente e estaticamente tipadas, como o *Go*, destacam-se em ambientes *serverless* por gerarem binários estáticos enxutos. Devido a essas características, aplicações em *Go* apresentam rápida inicialização e consumo reduzido de memória, tornando-se candidatas ideais para mitigar os efeitos negativos do *Cold Start* em sistemas distribuídos.

Por outro lado, o ecossistema *Java* tradicionalmente sofre com inicializações lentas e alto consumo de memória em ambientes *FaaS* devido à dependência estrutural da *JVM* e ao *overhead* da compilação *Just-In-Time (JIT)*, que ocorre em tempo de execução. Para adequar o *Java* a ambientes de nuvem restritos e otimizados para inicialização rápida, surgiram *frameworks* modernos como o *Quarkus*. A principal vantagem tecnológica do *Quarkus* reside no suporte à compilação *Ahead-of-Time (AOT)*, por meio do GraalVM. Diferente da abordagem tradicional *JIT*, a compilação *AOT* antecipa o processamento para a fase de *build* da aplicação. Durante essa etapa, realiza-se uma análise estática rigorosa que descarta códigos não utilizados e compila o código diretamente para linguagem da máquina. Essa transformação de aplicações *Java* em executáveis nativos elimina a necessidade de compilação dinâmica no momento da invocação, reduzindo drasticamente a latência de *startup* e permitindo que o ecossistema *Java* compita em eficiência com linguagens nativas como o *Go* no paradigma *FaaS* [Maksimov 2025].

3. Trabalhos Relacionados

A literatura recente tem dedicado considerável atenção aos desafios de desempenho e custo inerentes à arquitetura *serverless*, especialmente no que tange à mitigação do *Cold Start* e à otimização de recursos. Diante disso, esta seção apresenta estudos empíricos que investigam esses tópicos.

No contexto de estratégias de otimização e desempenho geral em ambientes *Serverless*, Bardsley et al. (2018) conduziram uma pesquisa focada na avaliação de técnicas

para mitigar a latência e melhorar a eficiência de execução. O estudo adotou uma metodologia empírica para testar diferentes configurações de dimensionamento e empacotamento de funções, revelando que o pré-aquecimento de instâncias e a redução do tamanho do pacote de implantação são cruciais para a diminuição da latência inicial. Embora o trabalho estabeleça uma base histórica importante sobre as melhores práticas de otimização serverless, ele antecede a maturidade de frameworks modernos de compilação nativa. O presente trabalho complementa essa visão ao investigar como inovações recentes, como, por exemplo, a compilação AOT via *GraalVM* no *Quarkus* se comparam a linguagens nativamente enxutas.

Avançando para avaliações contemporâneas de custo e latência sob condições variáveis, Munisif (2023) investigou o comportamento da *AWS Lambda* em cenários de alta escalabilidade. Utilizando testes de carga rigorosos, o autor avaliou como o aumento repentino de requisições impacta o faturamento e o tempo de resposta da plataforma. Os resultados demonstraram que, embora o escalonamento automático seja eficiente, picos de acesso não previstos geram instabilidades momentâneas de latência e aumentos não lineares nos custos devido à proliferação de execuções simultâneas. A pesquisa proposta neste trabalho relaciona-se diretamente com o estudo de Munisif (2023), adotando uma abordagem semelhante de injeção de carga. No entanto, enquanto Munisif foca na infraestrutura genérica da *AWS*, este TCC avança ao isolar o impacto do *runtime* nessas métricas, utilizando ferramentas como *K6* e *AWS X-Ray* para determinar qual tecnologia oferece melhor previsibilidade de custo e latência sob estresse.

A problemática específica do *Cold Start* foi empiricamente destrinchada por Raj et al. (2024), na qual eles propuseram um método de avaliação comparativa entre diferentes provedores de nuvem para mensurar o atraso na inicialização de funções a frio. Os dados reportados indicaram que o tempo de alocação do contêiner somado à inicialização do *runtime* da linguagem constitui o maior gargalo de desempenho em requisições assíncronas e esporádicas. Este estudo fornece um embasamento métrico fundamental para a compreensão do *Cold Start*. Dessa forma, este trabalho utilizará os mesmos princípios de medição para isolar essa latência, mas com o objetivo específico de contrastar o peso inerente da *JVM* frente à execução de binários estáticos em *Go*, respondendo se a barreira documentada pelos autores pode ser transposta por decisões arquiteturais de *software*.

Para estabelecer um referencial de alta performance em linguagens compiladas, Balla et al. (2020) investigaram o comportamento de diferentes *runtimes*, no caso *Go*, *Python* e *Node.js*, em plataformas *FaaS* de código aberto. A pesquisa adotou uma abordagem empírica, submetendo os *runtimes* a testes de carga com simulações de requisições de eco, processamento intensivo de CPU e operações com uso intensivo de dados. Os resultados evidenciaram que as funções desenvolvidas em *Go* superaram todas as outras variantes em termos de latência mediana, confirmando a eficiência do ecossistema para ambientes serverless dinâmicos. A inclusão deste estudo é tática para a presente pesquisa, pois consolida empiricamente o *Golang* como um *baseline* de alto desempenho contra o qual o framework *Quarkus* será avaliado. Ao utilizar esses dados do ecossistema *Go* como referência, o atual trabalho buscará responder de forma objetiva se as otimizações implementadas no *Java* conseguem alcançar uma eficiência semelhante durante a execução na plataforma *AWS Lambda*.

Por fim, lidando diretamente com as limitações do ecossistema *Java* em *FaaS*, Maksimov (2025) realizou uma análise detalhada da latência de inicialização utilizando *frameworks Java* modernos implantados na *AWS Lambda*. A pesquisa focou em mensurar o impacto de ferramentas que prometem otimização de *boot*, evidenciando que o uso de compilação nativa reduz drasticamente o *Cold Start* em comparação ao *Java* tradicional, aproximando-o de tempos de inicialização aceitáveis para aplicações síncronas. Sendo assim, o trabalho do autor evidenciou que o *Quarkus* apresentou um tempo de latência inferior em comparação aos demais *frameworks* avaliados. Contudo, enquanto o autor foca exclusivamente na comparação dentro do ecossistema *Java*, o presente estudo utilizará o *Go* como um referencial de comparação externo. A intenção é comparar se os ganhos reportados por Maksimov (2025) no *Quarkus* são suficientes para torná-lo financeiramente e computacionalmente competitivo quando colocado lado a lado com uma linguagem nativamente compilada.

4. Materiais e Métodos

Este estudo caracteriza-se como uma pesquisa de abordagem quantitativa e de caráter experimental. A dimensão quantitativa origina-se da coleta e análise de métricas objetivas, tais como latência de inicialização, tempo de execução e consumo e alocação de memória, utilizadas para mensurar o desempenho e o custo financeiro das tecnologias avaliadas.

O caráter experimental da pesquisa decorre da manipulação controlada das variáveis independentes, representadas pelas tecnologias utilizadas nas funções *serverless*, pela configuração de recursos computacionais e pelos diferentes níveis de carga aplicados durante os testes. A partir dessa manipulação, serão observados os impactos nas variáveis dependentes, relacionadas ao desempenho e ao consumo de recursos das aplicações. O experimento será conduzido em ambiente computacional controlado na nuvem, buscando assegurar reprodutibilidade, confiabilidade e redução de vieses decorrentes de interferências externas.

4.1. Metodologia Utilizada

A metodologia adotada foi estruturada em cinco etapas sequenciais, definidas com o objetivo de garantir padronização, reprodutibilidade e alinhamento direto com os objetivos específicos da pesquisa. O fluxo completo das etapas metodológicas propostas pode ser visualizado na Figura 1.



Figura 1. Metodologia

Etapa 1. Desenvolvimento das funções *Lambdas*: Inicialmente, serão implementadas funções *serverless* contendo regras de negócio equivalentes, assegurando o

mesmo nível de complexidade computacional entre ambas as implementações. O objetivo dessa equivalência é garantir que as diferenças observadas nos resultados estejam associadas exclusivamente às características das tecnologias avaliadas, e não às particularidades das regras de negócio implementadas. As funções serão projetadas para executar em três cenários distintos de processamento:

- **Processamento intensivo de CPU:** Implementação de um algoritmo de alto custo computacional isolado. Para este cenário, será utilizado o cálculo de fatoração de números primos em larga escala, visando estressar estritamente a capacidade de processamento e avaliar o tempo de execução bruto do *runtime* sem a interferência de latência de rede.
- **Execução com concorrência e paralelismo:** Simulação de uma carga de trabalho altamente paralela. Sendo assim, o teste consistirá no recebimento de um lote de dados que deverá ser dividido, processado e agrupado paralelamente. Para isso, serão utilizadas as bibliotecas de concorrência de cada linguagem. No caso do *Go* será utilizado o *Goroutines* e para o *Quarkus* será usado o *Java Virtual Threads*. Com isso, será possível avaliar a eficiência de cada modelo na orquestração de tarefas simultâneas.
- **Operações de entrada e saída (I/O):** Focado em requisições de rede, este cenário envolverá operações de leitura e escrita no banco de dados *Amazon DynamoDB*. O objetivo é medir a eficiência de como os runtimes gerenciam o consumo de memória e o tempo de bloqueio durante chamadas externas.

As implementações serão realizadas utilizando as tecnologias *Go* e *Quarkus*. A linguagem *Go* foi selecionada por possuir compilação nativa, baixo consumo de memória e reduzido tempo de inicialização, características frequentemente associadas a ambientes *serverless* de alta eficiência. Já o *framework Quarkus* foi escolhido por disponibilizar otimizações voltadas ao ecossistema *Java*, incluindo compilação *AOT* por meio do *GraalVM*, visando reduzir o tempo de inicialização e o consumo de recursos das aplicações, permitindo avaliar a viabilidade e o desempenho do ecossistema *Java* no paradigma *FaaS*.

Etapa 2. Provisionamento do ambiente de pesquisa: Com o objetivo de assegurar padronização e evitar inconsistências provenientes de configurações manuais, toda a infraestrutura utilizada no experimento será definida utilizando o paradigma *Infrastructure as Code (IaC)*, por meio da ferramenta *Terraform*. O processo de *build* e *deploy* das aplicações será automatizado utilizando *pipelines* de *CI/CD* com o *GitHub Actions*.

O ambiente experimental será provisionado na plataforma *AWS Lambda*. A escolha da *AWS* justifica-se pela ampla adoção do serviço no mercado de computação *serverless*, além da disponibilidade de ferramentas nativas de monitoramento, observabilidade e coleta de métricas detalhadas de execução e faturamento.

Etapa 3. Execução dos testes e injeção de carga: Após o provisionamento do ambiente, os testes serão conduzidos utilizando a ferramenta *K6*, especializada em testes de carga baseados em código. Essa ferramenta será responsável pela geração controlada de requisições às funções *serverless*, permitindo a simulação de diferentes cenários de utilização.

Para assegurar uma avaliação abrangente, os experimentos contemplarão perfis distintos de demanda. Por um lado, serão aplicados Testes de Pico (*Spike Testing*), intercalando períodos prolongados de inatividade com injeções abruptas de requisições, com

o objetivo de forçar o esgotamento das concorrências e induzir múltiplas ocorrências de *Cold Start*. Por outro lado, cenários de alta concorrência e tráfego contínuo serão simulados por meio de Testes de Carga (*Load Testing*), voltados à análise da estabilidade e da eficiência no consumo de recursos das funções operando sob a condição de *Warm Start*. Dessa forma, será possível avaliar o impacto e a viabilidade financeira das diferentes tecnologias em situações representativas de uso real.

Etapa 4. Coleta de dados: Durante a execução dos testes, as ferramentas de monitoramento e observabilidade nativas da plataforma, *AWS CloudWatch* e *AWS X-Ray*, serão empregadas como instrumentos de medição.

O *AWS CloudWatch* será empregado para coletar métricas relacionadas ao tempo total de execução, duração faturada, utilização máxima de memória e quantidade de invocações realizadas. Paralelamente, o *AWS X-Ray* será utilizado para realizar o *tracing* distribuído das requisições, permitindo identificar e isolar com maior precisão o tempo consumido especificamente durante o processo de inicialização das *Lambdas*.

Etapa 5. Análise dos resultados: Após a coleta, os dados serão organizados, tratados e submetidos a processos de limpeza e validação, visando remover inconsistências e garantir a integridade das análises estatísticas.

Posteriormente, será realizada uma análise estatística descritiva dos resultados, incluindo o cálculo de métricas como média, mediana e percentis (p90 e p99). Essas métricas permitirão comparar o comportamento das aplicações desenvolvidas em *Go* e *Quarkus* sob diferentes condições de carga, considerando tanto aspectos de desempenho quanto de custo financeiro.

A partir dessa análise, busca-se identificar padrões de comportamento relacionados à latência, ao consumo de recursos e ao custo das execuções, fornecendo subsídios quantitativos para avaliar a adequação de cada tecnologia ao contexto de aplicações *serverless* executadas na *AWS Lambda*.

4.2. Métricas

As métricas adotadas neste estudo foram definidas com o objetivo de avaliar, de forma quantitativa, tanto o desempenho computacional quanto o impacto financeiro das tecnologias analisadas no contexto do ecossistema *serverless*. A seleção dessas métricas busca fornecer uma análise abrangente sobre eficiência de execução e custo financeiro das aplicações implementadas

Para analisar o impacto do ambiente de execução durante a inicialização das funções, utilizou-se a métrica de Latência de *Cold Start*, mensurada em milissegundos (ms). Essa métrica corresponde ao tempo necessário para que a *AWS Lambda* realize o provisionamento do contêiner, inicialize o *runtime* da linguagem e prepare a aplicação para atender à primeira requisição após um período de inatividade. Sua utilização é fundamental para avaliar o impacto estrutural e o peso computacional de cada tecnologia em cenários de inicialização a frio.

Complementarmente, foi avaliado o Tempo de Execução em *Warm Start*, também mensurado em milissegundos (ms). Essa métrica representa o tempo efetivo de processamento da lógica de negócio quando a função já se encontra previamente inicializada e reutilizando um ambiente ativo. Dessa forma, torna-se possível analisar o comportamento

das aplicações em condições de operação contínua, reduzindo a influência do processo de inicialização.

Já no contexto de utilização de recursos computacionais, foi monitorado o Consumo Máximo de Memória, medido em *Megabytes* (MB). Essa métrica registra o maior volume de memória consumido durante a execução das funções, permitindo identificar o nível de eficiência no gerenciamento de recursos computacionais por cada tecnologia avaliada.

Por fim, foi considerada a métrica de Custo Financeiro Estimado, expressa em dólares americanos (USD). Esse indicador foi calculado com base no modelo de cobrança da *AWS Lambda*, que considera principalmente a quantidade de memória alocada e o tempo de execução das funções. A utilização dessa métrica possibilita extrapolar os resultados experimentais para cenários reais de utilização em larga escala, permitindo avaliar a viabilidade econômica de cada abordagem tecnológica em ambientes de produção.

4.3. Cronograma da Pesquisa

Para a execução das etapas experimentais e a consolidação dos resultados obtidos ao longo da pesquisa, foi elaborado um cronograma de atividades referente à disciplina de TCC II, compreendendo o período de agosto a novembro de 2026. O planejamento foi estruturado em ciclos quinzenais, permitindo uma distribuição organizada das atividades de desenvolvimento, execução dos experimentos, análise dos dados e produção textual do trabalho.

Considerando que a pesquisa possui caráter individual, o cronograma apresentado contempla as atividades desenvolvidas exclusivamente pelo autor, refletindo o planejamento necessário para garantir a condução sistemática do estudo, o cumprimento dos objetivos propostos e a finalização adequada do estudo dentro do período estabelecido.

Tabela 1. Distribuição de tarefas por datas

Tarefas	2026						
	Agosto		Setembro		Outubro		Novembro
	1ªQ	2ªQ	1ªQ	2ªQ	1ªQ	2ªQ	1ªQ
Desenvolvimento das Lambdas em Go e Quarkus	X	X					
Implementação dos recursos da AWS via IaC com Terraform			X				
Criação das pipelines CI/CD com GitHub Actions				X			
Deploy da infraestrutura na AWS				X			
Implementação dos testes de carga utilizando K6					X		
Execução de testes de carga					X		
Extração de resultados e dados de execução no AWS CloudWatch e no AWS X-Ray						X	
Obtenção e análise das métricas dos resultados obtidos.						X	
Discussão dos resultados encontrados e conclusão do estudo.							X

Referências

- Balla, D., Maliosz, M., Simon, C., and Gehberger, D. (2020). Tuning runtimes in open source faas. In *2020 43rd International Conference on Telecommunications and Signal Processing (TSP)*, pages 1–8. IEEE.
- Bardsley, D., Ryan, L., and Howard, J. (2018). Serverless performance and optimization strategies. In *2018 IEEE International Conference on Smart Cloud (SmartCloud)*, pages 19–26, New York, NY, USA. IEEE.
- Eivy, A. (2017). Be wary of the economics of ”serverless” cloud computing. *IEEE Cloud Computing*, 4(2):6–12.
- Fox, A., Griffith, R., Joseph, A., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., and Stoica, I. (2017). Serverless computing: Design, implementation, and performance. *IEEE Distributed Systems Online*, 18(10):1–10.
- Hassan, H. B., Barakat, S. A., and Sarhan, Q. I. (2021). Serverless computing: A survey of opportunities, challenges, and applications. *ACM Computing Surveys (CSUR)*, 54(8):1–40.
- Leitner, P., Wittern, E., Spillko, J., et al. (2019). A mixed-method empirical study of function-as-a-service software development in industrial practice. *Journal of Systems and Software*, 149:340–359.
- Maksimov, V. Y. (2025). Startup latency analysis in java frameworks for serverless aws lambda deployments. *The American Journal of Engineering and Technology*, 7(4):16–21.
- Männer, J., Endreß, M., Heckel, T., and Wirtz, G. (2018). Cold start influencing factors in serverless computing. In *2018 IEEE/ACM International Conference on Utility and Cloud Computing Companion (UCC Companion)*, pages 181–186. IEEE.
- Munisif, B. (2023). Serverless at scale: Evaluating aws lambda cost and latency under varying loads. *International Journal of Innovative Research in Science, Engineering and Technology*, 12.
- Raj, V., Chhapparwal, H., and Saale, S. (2024). Empirical evaluation of cold start latency in serverless platforms. In *2024 3rd International Conference for Advancement in Technology (ICONAT)*, pages 1–6, Goa, India. IEEE.
- van Eyk, E., Iosup, A., Seif, S., and Thömmes, M. (2018). Serverless computing: Economic and architectural impact. *Proceedings of the 2018 ACM on International Conference on Software Engineering*.
- Wang, L., Li, M., Zhang, Y., Ristenpart, T., and Swift, M. (2018). Peeking behind the curtains of serverless platforms. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 133–146.